

pybmc: A Python package for Bayesian model combination

Troy Dasher ^{1,2}, Pablo Giuliani ¹, Kyle Godbey ¹, and An Le^{1,2}✉

1 Facility for Rare Isotope Beams, Michigan State University, East Lansing, MI 48824, USA\\ 2

Department of Physics and Astronomy, Michigan State University, East Lansing, MI 48824, USA

Corresponding author

DOI: 10.xxxxxx/draft

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Open Journals](#)

Reviewers:

- @openjournals

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright¹⁷
and release the work under a
Creative Commons Attribution 4.0¹⁸
International License ([CC BY 4.0](#)).¹⁹

750-1750 words <https://joss.readthedocs.io/en/latest/paper.html> <https://joss.readthedocs.io/en/latest/ex>

Summary

Bayesian model combination (BMC) provides a principled framework for combining predictions from multiple models while quantifying uncertainty. Unlike simple averaging methods, BMC explicitly accounts for correlations among models and infers optimal model weights through Bayesian inference, producing predictions with calibrated uncertainty estimates.

This is a test edit to see the artifact (pdf paper)

The pybmc package implements this methodology in Python, allowing researchers to apply BMC to ensembles of predictive models defined on a shared domain. The package is designed for applications in which different theoretical or computational models produce systematically different predictions.

By providing a lightweight and flexible interface for constructing model ensembles and generating uncertainty-aware predictions, pybmc makes BMC accessible for research workflows. While motivated by applications in nuclear physics, the approach is general and applicable to a broad range of problems involving ensemble modeling and uncertainty-aware prediction.

Statement of need

Across diverse scientific disciplines, it is increasingly common to encounter problems that rely on different theoretical models, which often yield systematically different predictions for the same observable. In such settings, the challenge is not only to compare models, but to combine them in a statistically meaningful and reproducible way that accounts for correlations, redundancy, and uncertainty. BMC offers a useful framework to address this problem by constructing weighted predictions from multiple models using dimensionality reduction and Bayesian inference.

However, practical applications of BMC are limited by the lack of accessible and convenient software implementations. This software addresses this problem by providing a user-friendly, object-oriented framework Python package for BMC that supports the full workflow from aligned model tables to uncertainty-aware predictions. It integrates key steps such as data processing, dimensionality reduction (e.g., principal component analysis), and Bayesian statistical modeling into a coherent and reusable workflow. By organizing these operations into modular components, the package enables fast and simultaneous implementation of BMC across different datasets, while allowing flexible selection or extension of different components in the workflow.

State of the fields

This package represents the first user-focused software implementation of the model orthogonalization and combination strategy detailed in (Giuliani et al., 2024). While existing scientific Python tools such as scikit-learn (Pedregosa et al., 2011) and PyMC (Abril-Pla et al., 2023) provide components relevant to BMC, including dimensionality reduction and Bayesian inference, they do not offer an integrated implementation or any BMC workflow. There also exist other Bayesian model mixing software packages, including the Taweret software package (Ingles & others, 2024), but the current package is primarily designed to be used in scenarios where practitioners have precomputed databases of model predictions (with or without uncertainties) and wish to combine the results efficiently without needing to run additional simulations. This is particularly beneficial in nuclear physics, for instance, where global calculations of nuclear properties for all possible isotopes are computationally demanding.

Bayesian Model Combination

BMC provides a framework for constructing a single predictive model from a collection of m existing models $\mathcal{M}_k$, each producing predictions $M_k(x)$ for an observable defined over a common domain. The objective is to leverage the collective information contained in the ensemble while accounting for correlations and redundancies among models, which often arise when models share theoretical assumptions or calibration data.

We begin by assembling the predictions of the m models over n input locations into a matrix $X \in \mathbb{R}^{n \times m}$, where each column corresponds to a model and each row corresponds to a point in the domain. From this matrix, we define the ensemble mean prediction:

$$\phi_0(x_i) = \frac{1}{m} \sum_{k=1}^m M_k(x_i),$$

which captures the common structure shared across models. Subtracting this mean from the model predictions produces a centered representation that isolates the deviations between models.

To identify the dominant patterns in these deviations, we perform a singular value decomposition (SVD) of the centered matrix and retain a truncated set of components. This procedure yields an orthogonal basis $\phi_j(x)$ that spans the principal directions of variability among the models, with $j = 1, \dots, p$ with $p \leq m$ controlling the effective dimensionality of the representation. This step reduces the dimensionality of the problem and mitigates the impact of redundant or highly correlated models by filtering out directions associated with minor variations.

The combined model is then constructed as a linear expansion in principal components:

$$f^\dagger(x; \mathbf{b}) = \phi_0(x) + \sum_{j=1}^p b_j \phi_j(x),$$

where $\mathbf{b} = (b_1, \dots, b_p)$ are coefficients to be inferred from data. Because each basis function ϕ_j is itself a linear combination of the original models, this representation is itself a (restricted) combination of the model ensemble.

The relationship between the combined model and experimental observations y_i is described through $y_i = f^\dagger(x_i; \mathbf{b}) + \epsilon_i$, with $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, where σ represents the typical scale of the discrepancy between the model and the data and is inferred within the Bayesian framework.

Given a set of observations $\mathbf{y}$, the likelihood function takes the form:

$$p(\mathbf{y} | \mathbf{b}, \sigma) \propto \prod_{i=1}^n \frac{1}{\sigma} \exp \left(-\frac{(f^\dagger(x_i; \mathbf{b}) - y_i)^2}{2\sigma^2} \right).$$

Combining this likelihood with suitable prior distributions for $\mathbf{b}$ and σ yields the posterior distribution $p(\mathbf{b}, \sigma | \mathbf{y})$, which is sampled in pybmc using a Gibbs sampling procedure.

Predictions at new inputs are obtained by evaluating the combined model across posterior samples, resulting in a posterior predictive distribution:

$$p(y(x) | y) = \int p(y(x) | \mathbf{b}, \sigma), p(\mathbf{b}, \sigma | y), d\mathbf{b}, d\sigma,$$

from which summary statistics such as median predictions and credible intervals can be computed.

This formulation allows the BMC approach to reduce the effective dimensionality of the model space through orthogonalization, limit the impact of correlated or redundant models, and produce uncertainty-calibrated predictions. A more detailed discussion of the methodology can be found in (Giuliani et al., 2024).

Software design

Class architecture

pybmc is organized around two core classes that separate data management from Bayesian inference, supported by small utility modules that implement the numerical algorithms.

The `Dataset` class (module `pybmc.data`) encapsulates all data access and preprocessing. It loads model outputs and (optionally) experimental truth values from HDF5 or CSV files into synchronized, pandas DataFrames. Users specify a list of model identifiers, a list of physical properties (e.g., binding energy or charge radius), and the domain columns (e.g., nucleon numbers N and Z). `Dataset.load_data` then aligns the models on their common domain and, when a `truth_column_name` is supplied, left-joins the truth data so that experimental information can live on a smaller domain than the model ensemble. Additional methods provide convenient views of the loaded data, domain-aware filtering, and utilities for constructing training/validation/test splits.

The `BayesianModelCombination` class (module `pybmc.bmc`) implements the BMC workflow. It operates on the synchronized DataFrames produced by `Dataset` and keeps track of the list of models to be combined and the column containing the experimental truth. The class exposes three main operations: `orthogonalize`, which constructs an orthogonal basis of model differences by applying singular value decomposition (SVD) to centered model predictions; `train`, which performs Gibbs sampling on the truncated SVD coefficients to obtain posterior samples of the combination weights and residual uncertainty; and `predict / evaluate`, which turn those samples into posterior predictive draws, credible intervals, and coverage diagnostics.

Low-level numerical routines are factored into the `pybmc.inference_utils` and `pybmc.sampling_utils` modules. The former provides SVD post-processing (`USVt_hat_extraction`) and a Gibbs sampler for Bayesian linear regression, while the latter provides utilities to turn posterior samples into predictive draws and credible intervals (`rndm_m_random_calculator`) and to compute coverage statistics (`coverage`). Keeping these routines outside the main classes keeps the public API small and makes it straightforward to swap or extend samplers and diagnostic tools without altering the high-level workflow.

Workflow pipeline

The overall workflow pipeline is designed to mirror typical scientific usage, starting from precomputed model tables and ending with uncertainty-quantified predictions and diagnostics.

1. **Data loading and alignment.** The user constructs a `Dataset` pointing to a single HDF5 or CSV file that contains multiple predictive models and, optionally, an experimental or “truth” column. Calling `Dataset.load_data` with a list of model names, property keys, and domain columns returns a dictionary of aligned DataFrames, one per property, with columns for the domain and each model. When a truth column is specified, it is left-joined to the common model domain so that the training set can be restricted to

the subset of points with experimental data while still enabling prediction across the full model domain.

2. **Domain selection and splitting.** The same Dataset instance provides tools to select physically relevant subsets (for example, restricting to a range in N and Z) and to partition the data into training, validation, and test sets. The `split_data` method supports both simple random splits and an “inside-to-outside” algorithm based on distances in the (N, Z) plane, which is useful in nuclear physics applications where one wishes to train near stability and test predictions further from known nuclei. This design keeps domain knowledge and splitting strategies in the data layer rather than in the inference layer.
3. **Orthogonalization of model predictions.** The user then initializes a `BayesianModelCombination` instance with the list of models to be combined, the data dictionary returned by Dataset, and the name of the truth column. The `orthogonalize` method centers each model's predictions around the ensemble mean and applies SVD to obtain an orthogonal basis of model differences. The user controls the effective model complexity through a `components_kept` parameter, which specifies how many singular vectors to retain. This step improves numerical stability and makes the subsequent Bayesian inference operate on a low-dimensional set of latent components rather than on the raw model table.
4. **Bayesian inference.** Given the orthogonalized design matrix, the `train` method runs a Gibbs sampler (implemented in `pybmc.inference_utils.gibbs_sampler`) to draw posterior samples for the combination weights and a residual noise scale. The method accepts a `training_options` dictionary that allows users to control the number of iterations and the prior hyperparameters for the weights and variance. This separation between high-level workflow (`BayesianModelCombination.train`) and low-level Markov Chain Monte Carlo implementation keeps the public interface simple while leaving room for more advanced samplers.
5. **Prediction and diagnostics.** Once the sampler has converged, the `predict` method projects the posterior weights back onto the original model space to generate posterior predictive draws for any property present in the original data dictionary, returning both the raw draws and summary DataFrames with median predictions and 95% credible intervals. The `evaluate` method uses the same predictive draws to compute empirical coverage over a grid of credible interval widths, providing a simple, model-agnostic diagnostic of how well the BMC captures the experimental truth where it is available.

This layered architecture—data access and domain logic in Dataset, orthogonalization and sampling in BayesianModelCombination, and numerical routines in focused helper modules—was chosen to reflect how practitioners already work with precomputed model tables while making each step of the combination pipeline explicit, testable, and replaceable.

Users can adjust the workflow at several points. For example, users can modify or extend the sampling routines in `pybmc.inference_utils` (for example, to use the provided simplex-constrained Gibbs sampler or custom MCMC kernels). The package is deliberately designed to be domain-agnostic, so that the same architecture can be applied to any application where practitioners need to combine multiple, potentially correlated predictive models into a single, uncertainty-quantified prediction.

More detailed tutorials, additional examples, and a complete API reference are available in the online documentation:

<https://ascsn.github.io/pybmc/docs>

Research impact statement

The original methodology was published in a peer-reviewed journal (Giuliani et al., 2024), and the first application of the methodology has also recently been published in an analysis of

new experimental data from the Facility for Rare Isotope Beams (Ireland et al., 2026). Once the package was completed, it began replacing the original, bespoke implementation in new scientific works, including a study on Q-alpha trends in superheavy nuclei, the impact of nuclear uncertainties in r-process nucleosynthesis, and a systematic study of charge radii. Students involved in these projects have presented results obtained using the software at the American Physical Society's annual Division of Nuclear Physics meeting and at presentations locally. The package has also been adopted for release with the Bayesian Analysis for Nuclear Dynamics collaboration's V0.5 release and disseminated broadly to the nuclear physics community.

AI usage disclosure

Generative AI was used in the creation of a package template early in the development process and during development via inline autocomplete within the developers' editors. No code was adopted without human oversight, and our metric for correctness was defined by detailed numerical comparison to the bespoke implementation already published in [cite]. AI tools were also used to expand the package documentation and write additional tests, with the resulting documentation going through a round of human review for completeness, correctness, and accessibility.

Acknowledgments

Citations

- Abriel-Pla, O., Andreani, V., Carroll, C., Dong, L., Fannesbeck, C. J., Kochurov, M., Kumar, R., Lao, J., Luhmann, C. C., Martin, O. A., Osthege, M., Vieira, R., Wiecki, T., & Zinkov, R. (2023). PyMC: A modern and comprehensive probabilistic programming framework in Python. *PeerJ Computer Science*, 9(e1516). <https://doi.org/10.7717/peerj-cs.1516>
- Giuliani, P., Godbey, K., Kejzlar, V., & Nazarewicz, W. (2024). Model orthogonalization and bayesian forecast mixing via principal component analysis. *Phys. Rev. Res.*, 6, 033266. <https://doi.org/10.1103/PhysRevResearch.6.033266>
- Ingles, K., & others. (2024). Taweret: A python package for bayesian model mixing. *Journal of Open Source Software*, 9(97), 6175. <https://doi.org/10.21105/joss.06175>
- Ireland, C., Bollen, G., Campbell, S., Chen, X., Erington, H., Gamage, N., Godbey, K., Houff, A., Izzo, C., Knight, B., Lalit, S., Leistenschneider, E., Lykiardopoulou, E., Maier, F., Nazarewicz, W., Orford, R., Porter, W., Quick, C., Ravlic, A., & Villari, A. (2026). Mass of sn 101 and bayesian extrapolations to the proton drip line. *Physical Review C*, 113. <https://doi.org/10.1103/vck7-1c4t>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.